

CA-7

Que 1:

explanation of output: we have two thread which have their own local var and one global var, the address of global var is same for main process and both threads where as local variables have different addresses

CODE:

```
#include<pthread.h>
#include<stdio.h>

int global_var;

void *print_add(void* arg){
int thread_local;
int id=*(int*)arg;
printf("Thread ID: %d and it's local address: %p, global add = %p\n",id,(void*)&thread_local,
(void*)&global_var);
return NULL;
}

int main(){
int main_local;
pthread_t tid1;
pthread_t tid2;
int id1=1,id2=2;

pthread_create(&tid1,NULL,print_add,&id1);
pthread_create(&tid2,NULL,print_add,&id2);

pthread_join(tid1,NULL);
pthread_join(tid2,NULL);

printf("Main: Local add=%p, global add =%p\n",(void*)&main_local, (void*)&global_var);

return 0;
}
```

```
harshita-patidar@harshita-patidar-HP-Pavilion-Laptop-15-eg3xxx:~/Academics/Sem5/OS/CA7$ gcc que1.c -lpthread
harshita-patidar@harshita-patidar-HP-Pavilion-Laptop-15-eg3xxx:~/Academics/Sem5/OS/CA7$ ./a.out

Thread ID: 2 and it's local address: 0x72e0991fdeb0, global add = 0x5d28f8ce5014
Thread ID: 1 and it's local address: 0x72e0999feeb0, global add = 0x5d28f8ce5014
Main: Local add=0x7ffe81dbf19c, global add =0x5d28f8ce5014
```

Que 2:

explanation of output: in this, 10 threads are created, each of which prints its own unique ID from 1 to 10. The main program then waits for all ten threads to complete their execution before printing a final message and terminating.

CODE:

```

#include<pthread.h>
#include<stdio.h>

void *print_id(void* arg){
int id=*(int*)arg;
printf("Thread %d running!\n",id);
return NULL;
}

int main(){
pthread_t tids[10];
pthread_t main_tid;

int ids[10];

for(int i=0;i<10;i++)
{
ids[i]=i+1;
pthread_create(&tids[i],NULL,print_id,&ids[i]);
}
for(int i=0;i<10;i++)
{
pthread_join(tids[i],NULL);
}

printf("ALL THE THREADS ARE DONE!\n");

return 0;
}

```

```

harshita-patidar@harshita-patidar-HP-Pavilion-Laptop-15-eg3xxx:~/Academics/Sem5/OS/CA7$ gcc que2.c -lpthread
harshita-patidar@harshita-patidar-HP-Pavilion-Laptop-15-eg3xxx:~/Academics/Sem5/OS/CA7$ ./a.out
Thread 1 running!
Thread 4 running!
Thread 2 running!
Thread 5 running!
Thread 3 running!
Thread 6 running!
Thread 7 running!
Thread 8 running!
Thread 9 running!
Thread 10 running!
ALL THE THREADS ARE DONE!

```

Que 3:

explanation of output:

q. why final value differ from expected output?: because increment is not a atomic operation and all 10 threads are again and again getting scheduled and descheduled, so before some thread store the incremented value, the other thread is fetching the val of counter.

q. what is race condition and how it affects this program? A race condition occurs when two or more threads access a shared resource (like a global variable) at the same time, and at least one of them modifies it, while the final outcome depends on the *timing* or *interleaving* of their operations. Because thread execution order is unpredictable, the result becomes non-deterministic, it may

change each time you run the program.

CODE:

```
#include<pthread.h>
#include<stdio.h>
int counter=0;

void *func(void* arg){
for(int i=0;i<1000000;i++)
{
    counter+=1;
}
return NULL;
}

int main(){
pthread_t tids[10];

for(int i=0;i<10;i++)
{
    pthread_create(&tids[i],NULL,func,NULL);
}
for(int i=0;i<10;i++)
{
    pthread_join(tids[i],NULL);
}

printf("FINAL VALUE OF COUNTER IS: %d!\n",counter);

return 0;
}
```

```
harshita-patidar@harshita-patidar-HP-Pavilion-Laptop-15-eg3xxx:~/Academics/Sem5/OS/CA7$ touch que3.c
harshita-patidar@harshita-patidar-HP-Pavilion-Laptop-15-eg3xxx:~/Academics/Sem5/OS/CA7$ gcc que3.c -lpthread
harshita-patidar@harshita-patidar-HP-Pavilion-Laptop-15-eg3xxx:~/Academics/Sem5/OS/CA7$ ./a.out
FINAL VALUE OF COUNTER IS: 2292859!
harshita-patidar@harshita-patidar-HP-Pavilion-Laptop-15-eg3xxx:~/Academics/Sem5/OS/CA7$ ./a.out
FINAL VALUE OF COUNTER IS: 1901548!
harshita-patidar@harshita-patidar-HP-Pavilion-Laptop-15-eg3xxx:~/Academics/Sem5/OS/CA7$ ./a.out
FINAL VALUE OF COUNTER IS: 1657850!
harshita-patidar@harshita-patidar-HP-Pavilion-Laptop-15-eg3xxx:~/Academics/Sem5/OS/CA7$ ./a.out
FINAL VALUE OF COUNTER IS: 1576728!
harshita-patidar@harshita-patidar-HP-Pavilion-Laptop-15-eg3xxx:~/Academics/Sem5/OS/CA7$ ./a.out
FINAL VALUE OF COUNTER IS: 1655112!
harshita-patidar@harshita-patidar-HP-Pavilion-Laptop-15-eg3xxx:~/Academics/Sem5/OS/CA7$ ./a.out
FINAL VALUE OF COUNTER IS: 1687750!
harshita-patidar@harshita-patidar-HP-Pavilion-Laptop-15-eg3xxx:~/Academics/Sem5/OS/CA7$ ./a.out
FINAL VALUE OF COUNTER IS: 1528597!
harshita-patidar@harshita-patidar-HP-Pavilion-Laptop-15-eg3xxx:~/Academics/Sem5/OS/CA7$ ./a.out
FINAL VALUE OF COUNTER IS: 1471440!
harshita-patidar@harshita-patidar-HP-Pavilion-Laptop-15-eg3xxx:~/Academics/Sem5/OS/CA7$ ./a.out
FINAL VALUE OF COUNTER IS: 1555463!
harshita-patidar@harshita-patidar-HP-Pavilion-Laptop-15-eg3xxx:~/Academics/Sem5/OS/CA7$
```

Que 4:

explanation: This program creates a single thread and passes it an integer (10) to calculate its square. The thread allocates memory on the heap for the result, returns a pointer to it, and the main thread retrieves this pointer via `pthread_join` to print the final value (100).

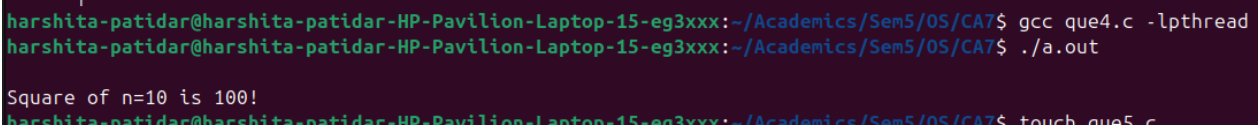
CODE:

```
#include<pthread.h>
#include<stdio.h>
#include<stdlib.h>
void *func(void* arg){
    int val=*(int*)arg;
    int* sq=malloc(sizeof(int));
    *sq=val*val;
    return sq;
}

int main(){

    pthread_t tid;
    int n=10;
    void* result;
    pthread_create(&tid,NULL,func,&n);
    pthread_join(tid,&result);
    printf("Square of n=%d is %d!\n", n, *(int*)result);

    return 0;
}
```



```
harshita-patidar@harshita-patidar-HP-Pavilion-Laptop-15-eg3xxx:~/Academics/Sem5/OS/CA7$ gcc que4.c -lpthread
harshita-patidar@harshita-patidar-HP-Pavilion-Laptop-15-eg3xxx:~/Academics/Sem5/OS/CA7$ ./a.out
Square of n=10 is 100!
harshita-patidar@harshita-patidar-HP-Pavilion-Laptop-15-eg3xxx:~/Academics/Sem5/OS/CA7$ touch que5.c
```

Que5:

explanation: This program asks the user for a number of threads and a list of integers, then concurrently calculates the square of each integer in a separate thread. After all threads have returned their results, the main program collects these squared values and prints their final sum.

CODE:

```
#include<pthread.h>
```

```

#include<stdio.h>
#include<stdlib.h>
void *func(void* arg){
    int val=*(int*)arg;
    int* sq=malloc(sizeof(int));
    *sq=val*val;
    return sq;
}

int main(){

    int n;
    printf("Enter no. of threads:");
    scanf("%d",&n);
    pthread_t tids[n];
    int vals[n];
    printf("Enter the values for each thread:\n");

    for(int i=0;i<n;i++)
    {scanf("%d",&vals[i]);}

    for(int i=0;i<n;i++)
    {
        pthread_create(&tids[i],NULL,func,&vals[i]);
    }
    void* result;
    int ans[n];
    for(int i=0;i<n;i++)
    {
        pthread_join(tids[i],&result);
        ans[i]=*(int*)result;
        printf("Thread %d returned %d\n",i+1,ans[i]);
    }
    int sum=0;
    for(int i=0;i<n;i++)
    { sum+=ans[i];}

    printf("Sum is %d!\n", sum);

    return 0;
}

```

```

harshita-patidar@harshita-patidar-HP-Pavilion-Laptop-15-eg3xxx:~/Academics/Sem5/OS/CA7$ gcc que5.c -lpthread
harshita-patidar@harshita-patidar-HP-Pavilion-Laptop-15-eg3xxx:~/Academics/Sem5/OS/CA7$ ./a.out
Enter no. of threads:5
Enter the values for each thread:
1
2
3
4
5
Thread 1 returned 1
Thread 2 returned 4
Thread 3 returned 9
Thread 4 returned 16
Thread 5 returned 25
Sum is 55!

```